

WSI ćwiczenie 2 - Algorytmy ewolucyjne i genetyczne

Natasza Jarecka

2025

1 Opis algorytmu

Algorytm genetyczny to algorytm mający za zadanie wyszukanie najlepszego rozwiązania, inspirowany zjawiskiem ewolucji biologicznej.

W poniższym zadaniu zaimplementowany został algorytm genetyczny z mutacją, selekcją ruletkową, krzyżowaniem jednopunktowym oraz sukcesją generacyjną.

Kolejne kroki algorytmu:

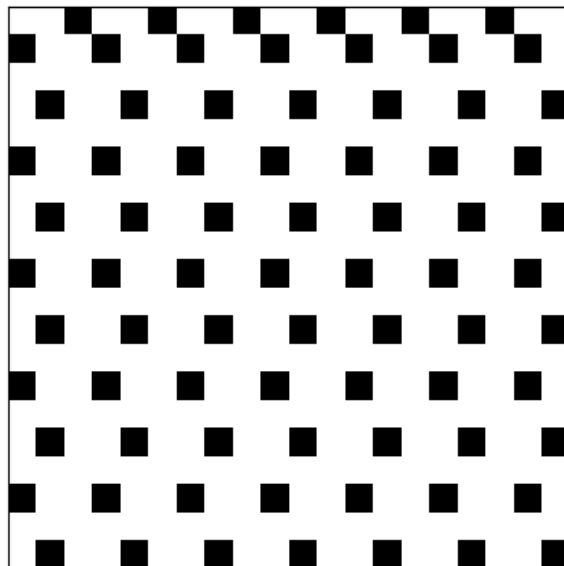
- Losowana jest pewna populacja początkowa
- Poprzez selekcję ruletkową wybierane są dwa osobniki
- Odbywa się krzyżowanie jednopunktowe (punkt "przecięcia" genotypu jest losowy)
- Powstają dwa nowe osobniki, połączenie "przeciętych" genotypów rodziców
- Nowe osobniki mutują z zadanyim prawdopodobieństwem
- Dodawane są do nowej populacji
- Nowa populacja zastępuje starą, proces powtarza się dopóki nie zostanie osiągnięta zadana liczba pokoleń
- Z ostatecznej populacji wybierany jest najlepszy osobnik

2 Rozwiązywany problem

Na planszy 20 x 20 rozmieszczane są obiekty. Każdy obiekt zajmuje jedno pole 1 x 1. Jeżeli obok obiektu znajduje się puste pole (nie bierzemy pod uwagę pól sąsiadujących po skosie) daje ono 1 punkt. W innych przypadkach pola dają 0 punktów. Zadaniem algorytmu jest znalezienie takiej planszy z rozmieszczonymi obiektami, która będzie dawała jak najwięcej punktów.

Rozwiązanie trywialne

Wyniki mojego algorytmu będę porównywać do mojego własnego rozwiązania, które wydaje mi się najlepsze. Rozmieszczenie obiektów na takiej planszy przedstawione jest poniżej.



czarny - pola z obiektami, biały - puste pola

Rozwiązanie to daje 269 punktów.

Jeżeli chodzi o górne ograniczenie wyniku, bezpiecznie jest powiedzieć, że nigdy liczba punktów nie będzie większa niż 400, jednak jest bardzo niedokładne ograniczenie. Myślę, że maksymalny wynik może być nieco większy lub równy 269.

3 Implementacja

Osobnik jest reprezentowany przez klasę `Individual`, której atrybutami są:

- `genotype` - genotyp osobnika, wektor zer i jedynek
- `mutation_prob` - prawdopodobieństwo mutacji

Klasa ma także dwie metody:

- `reproduce()` - jako argument przyjmuje innego osobnika, pojawia się krzyżowanie jednopunktowe z losowym punktem, zwraca dwa osobniki będące połączeniem genotypów rodziców
- `mutate()` - z zadanyam prawdopodobieństwem zmienia każdy z genów

Funkcja `genetic_algorithm()` przyjmuje jako argumenty:

- `evaluation` - funkcję ewaluacji osobników

- `gene_count` - liczba genów w genotypie
- `size` - rozmiar optymalizowanej, kwadratowej planszy
- `population_count` - liczba osobników w populacji
- `mutation_prob` - prawdopodobieństwo mutacji osobników
- `generations` - liczba pokoleń populacji

Na początku tworzona jest populacja losowych osobników za pomocą funkcji `generate_population()`. W pętli `for` wybierana jest dwójka rodziców przez funkcję `roulette_selection()`. Metodą `reproduce()` rodzice "rodzą" dwójkę dzieci. Dzieci poddawane są losowej mutacji (która może nie zajść w ogóle) i dodawane do nowej populacji. Zwiększa się licznik iteracji, a pętla `for` wykona się tyle razy, ile wynosi zadana liczba pokoleń podzielona na dwa (dzielenie całkowitoliczbowe). Jeżeli zadana populacja jest wartością nieparzystą, na koniec dodatkowa para rodziców urodzi dwójkę dzieci, ale do populacji zostanie dodane tylko jedno.

Z ostatniej populacji wybierany i zwracany jest najlepszy osobnik, za pomocą funkcji `find_the_best()`.

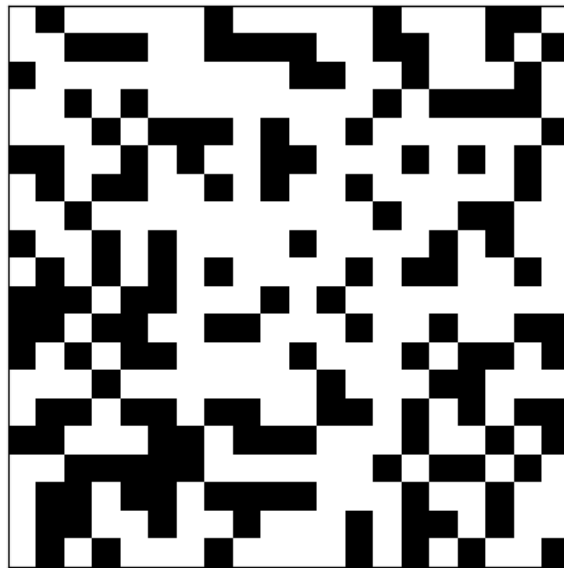
4 Eksperymenty i wyniki

Rozwiązanie losowe

Średnia arytmetyczna 100 rozwiązań losowych jest równa 184 z odchyleniem standardowym wynoszącym 7.

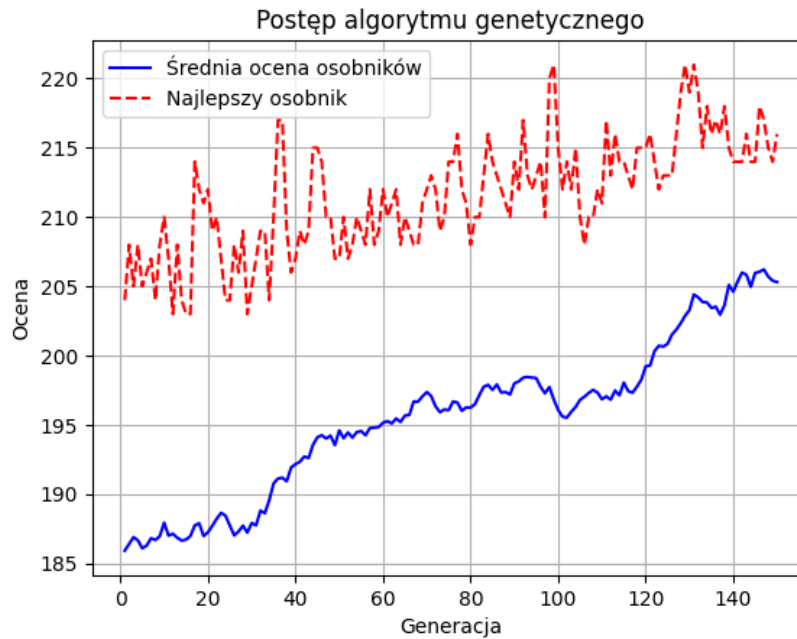
Najlepsze uzyskane rozwiązanie

Najlepszym rozwiązaniem jakie udało mi się uzyskać w trakcie eksperymentów jest plansza z wynikiem 222.



wartości parametrów:

- wielkość populacji = 150
- liczba pokoleń = 150
- prawdopodobieństwo mutacji = 0.001



Postęp algorytmu dla powyższych parametrów

Wpływ wielkości populacji

Dla liczby pokoleń równej 100 i prawdopodobieństwa mutacji równego 0.01. Średnia wyciągnięta jest z 10 wyników.

wielkość populacji	średni wynik	odchylenie standardowe
30	199.2	6.7
100	209.7	2.3
150	214.6	4.7

Wpływ prawdopodobieństwa mutacji

Przy populacji równej 100 i 100 pokoleniach.

prawdopodobieństwo mutacji	średni wynik	odchylenie standardowe
0.0001	210.4	3
0.01	209.7	2.3
0.1	204.3	1.8
0.8	202.9	2.7

Wpływ liczby generacji

Przy populacji równej 100 i prawdopodobieństwie mutacji równym 0.01.

liczba pokoleń	średni wynik	odchylenie standardowe
50	208.1	4.8
100	209.7	2.3
150	212.7	6.2

5 Wnioski

Z eksperymentów wynika, że im większy rozmiar populacji, tym lepszy wynik osiąga algorytm. Większa populacja wprowadza większą różnorodność, ale wykonanie się algorytmu zajmuje więcej czasu.

Jeżeli chodzi o prawdopodobieństwo mutacji najlepiej sprawdza się liczba rzędu 0.0001, co jest zgodne z tezą, że parametr ten powinien być równy odwrotności wymiarowości zadania. Zadany problem nie ma minimów lokalnych, więc mutacja nie jest pomocna w tym kontekście. Gdy geny częściej mutują, możemy zgubić dobre rozwiązanie.

Większa liczba pokoleń daje większą szansę na osiągnięcie dobrego rozwiązania, jednak jest bardziej kosztowna obliczeniowo.